Laboratory of Mobile Operating Systems

Introduction to Android development

Introduction

The goal of this exercise is to grasp the very basics of Android software development, such as:

- Android Studio and Android SDK;
- UI design;
- Multithreading in Kotlin.

Task 0.

Launch Android Studio and create a new project based on the Empty Views Activity template. While the project is being built, have a quick look at the contents of the following files:

- java.*your.namespace*.**MainActivity**, which is responsible for all logic happening in the main window;
- res.layout.**activity_main.xml**, which contains UI components used by MainActivity;
- **build.gradle** for the *app* module, which contains major build settings of your app, such as targeted SDK versions, compile options and various dependencies.
  If your initial project cannot be built, this file is most likely the root of all evil.

Finally, try to start your application on a device running on Android 11 or newer:

- *For physical devices:* make sure your phone or tablet has the "Enable Debugging" option turned on. Otherwise it won't be recognized as a compatible device.
- *For emulated devices:* go to Tools → Device Manager to see the list of available virtual devices. If you don't have a compatible device, create one. If you have one that stopped working correctly, try wiping its data.

Task 1. (1 point)

Add a new button with a click listener. First, open your layout file (activity_main.xml) in editor and add a *Button* component. You might also want to reposition the default *TextView* by removing some of its constraints. When you're done, open your MainActivity class and add a click listener to your button. Below is a sample Kotlin code to do this for you (make sure the following code happens after the setContentView() method is called):

```
val startButton = findViewById<Button>(R.id.yourButtonID)
startButton.setOnClickListener {
```

```
        // Add your logic here.
    }
```

Note: the Alt+Enter keyboard shortcut will be a big help for missing imports.

Finally, use what you've already learned to make sure that clicking the button actually makes some visible change in your application. This can be easily achieved by connecting to a TextView component and changing its *text* field:

```
yourTextView.text = "Click!";
```

Task 2. (2 points)

Create a background job that starts asynchronous work on a button click and logs its progress without blocking the UI thread. It would be a good idea to create a coroutine for this:

```
val job = lifecycleScope.launch {
    Log.d("YourTAG", "Job started")
    // Do some time-consuming calculation here.
    // Or just call the delay() method.
}
```

Note: to use the *lifecycleScope*, you will need to add the following dependency to your app module's build.gradle file:

```
implementation ("androidx.lifecycle:lifecycle-runtime-ktx:2.7.0")
```

Alternatively, if you don't like the coroutines way, you can still use an old-fashioned Java Thread instead:

```
val thread = Thread {
    Log.d("YourTAG", "Thread started")
    // Do some time-consuming calculation here.
    // Or just call the Thread.sleep() method.
}
thread?.start()
```

You can find your logs by opening the **Logcat** window in Android Studio.

Task 3. (1 point)

Update the background job to notify about its progress inside a TextView. Each notification requires using the UI thread for a short moment. Inside a coroutine job, the code can look something like this:

```
launch(Dispatchers.Main) {
    yourTextView.text = "Current progress: ${yourVariable}."
}
```

In case of a Java thread, it can look like this:

```
runOnUiThread {
    yourTextView.text = "Current progress: ${yourVariable}."
}
```

Task 4. (1 point)

Variant I

Add a ProgressBar to your app and make it reflect the current progress of the background job.

Variant II

Add a SeekBar to you app and make it control the total time required to finish the background job.